RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105

MA23434

OPTIMIZATION TECHNIQUES FOR AI

LABORATORY LAB MANUAL

Name : PREDHEP RAJ V

Year / Branch / Section : II YR / AIDS

Register No. : 241801215

Semester : IV SEMESTER

Academic Year : 2025-2026

INDEX

S.NO.	DATE	TITLE	PG NO	TEACHER'S SIGNATURE REMARKS
1 A)		LINEAR PROGRAMMING PROBLEM-CONSTRAINT OPTIMIZATION	1	
1 B)		GRAPHICAL METHOD	6	
2		TRANSPORTATION PROBLEM	11	
3		ASSIGNMENT PROBLEM	17	
4		INTEGER PROGRAMMING PROBLEM-BRANCH AND BOUND METHOD	20	
5		DYNAMIC PROGRAMMING – KNAPSACK PROBLEM, SUBSET SUM PROBLEM, LONGEST COMMON SUBSEQUENCE PROBLEM	25	
6		GRADIENT DESCENT METHOD-STOCHASTIC GRADIENT DESCENT ALGORITHM	31	
7		UNCONSTRAINED OPTIMIZATION-NONLINEAR LEAST SQUARES	34	
8		KUHN-TUCKER CONDITIONS - LAGRANGIAN MULTIPLIER METHOD	38	
9		CPM -ANALYSIS	41	
10		PERT -ANALYSIS	44	

EXPT NO: 1 A)	LINEAR PROGRAMMING PROBLEM-CONSTRAINT OPTIMIZATION
DATE:	

AIM:

To solve a Linear Programming Problem (LPP) for constraint optimization using Python.

PROCEDURE:

1. **Install Required Libraries:**
o Make sure you have numpy and scipy installed. You can install them using pip if not already installed.
bash
Copy code
pip install numpy scipy

2. **Define the Problem:**
o Identify the objective function to maximize or minimize.
o Determine the constraints (equality and inequality).

3. **Set Up the Problem in Python:**
o Use the scipy.optimize.linprog function to set up and solve the LPP.

4. **Run the Code:**
o Execute the Python code to get the optimized result.

5. **Analyze the Output:**
o Interpret the results and validate the solution against the problem constraints

CODE

PROGRAM 1:

```
import numpy as np
from scipy.optimize import linprog

# Coefficients of the objective function (maximize 3x + 2y → minimize -3x - 2y)
c = [-3, -2]

# Inequality constraints (Ax ≤ b)
A = [
    [2, 1],
    [4, -5]
]
b = [20, 10]

# Bounds for x and y (x ≥ 0, y ≥ 0)
x_bounds = (0, None)
y_bounds = (0, None)

# Solve the linear programming problem
result = linprog(c, A_ub=A, b_ub=b, bounds=[x_bounds, y_bounds], method='highs')

# Print results
print("Optimal value:", -result.fun)
print("Values of x:", result.x)
```

OUTPUT

```
Optimal value: 40.0
Values of x: [ 0. 20.]
```

Problem 2:

```
import numpy as np
from scipy.optimize import linprog

# Coefficients of the objective function (Minimize 2x + 3y)
c = [2, 3]
```

```
# Convert ≥ constraints into ≤ by multiplying by -1
A = [
    [-1, -1],
    [-2, -1]
]
b = [-4, -5]

# Bounds for x and y (x ≥ 0, y ≥ 0)
x_bounds = (0, None)
y_bounds = (0, None)

# Solve the linear programming problem
result = linprog(c, A_ub=A, b_ub=b, bounds=[x_bounds, y_bounds], method='highs')

# Print results
print("Minimum value:", result.fun)
print("Values of x:", result.x)
```

OUTPUT

Minimum value: 10.0

Values of x: [1. 3.]

RESULT:

The Linear Programming Problem Has Been Completed Sucessfully

EXPT NO: 1 B)	GRAPHICAL METHOD
DATE:	

AIM:

To solve a Linear Programming Problem (LPP) using the graphical method and find the optimal solution using Python.

PROCEDURE:

- Import required libraries: numpy, matplotlib, and scipy.optimize.
- Define the objective function coefficients. Since linprog performs minimization, convert the maximization problem into minimization by taking negative values.
- Define the inequality constraints in matrix form (A and b).
- Specify the bounds for decision variables ($x \geq 0$, $y \geq 0$).
- Use the linprog() function to solve the LPP.
- Extract the optimal values of decision variables (x and y).
- Convert the result back to maximization to obtain the optimal value of Z.
- Generate values for x to plot constraint equations.
- Plot all constraint lines on a graph.
- Shade the feasible region that satisfies all constraints.
- Mark the optimal solution point on the graph.
- Label axes, add legend, grid, and display the graph.

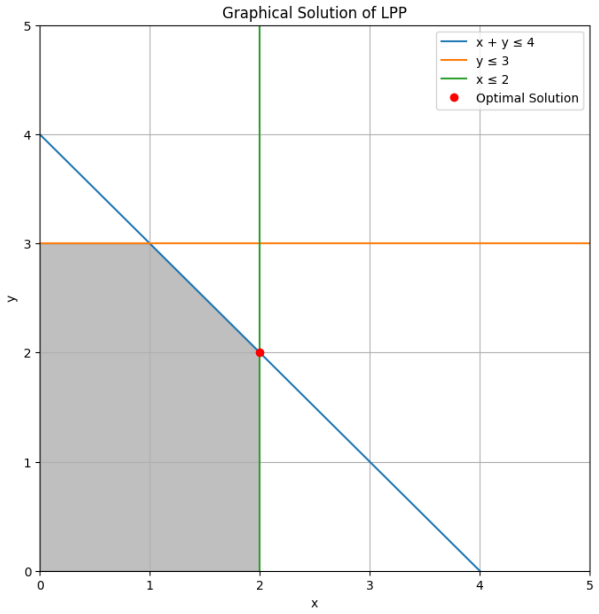
PROGRAM 1:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import linprog
# Objective function (maximize Z = 3x + 2y → convert to minimize)
c = [-3, -2]
# Inequality constraints (Ax ≤ b)
A = [
    [1, 1],   # x + y ≤ 4
    [0, 1],   # y ≤ 3
    [1, 0]    # x ≤ 2
]
b = [4, 3, 2]
# Bounds for x and y
x_bounds = (0, None)
y_bounds = (0, None)
# Solve using linprog
res = linprog(c, A_ub=A, b_ub=b, bounds=[x_bounds, y_bounds], method='highs')
# Extract solution
x_opt, y_opt = res.x
z_opt = -res.fun  # Convert back to maximization
print(f"Optimal solution: x = {x_opt}, y = {y_opt}, Z = {z_opt}")

# Plotting
x = np.linspace(0, 5, 400)
y1 = 4 - x
y2 = np.ones_like(x) * 3
y3 = np.ones_like(x) * 2
plt.figure(figsize=(8, 8))
plt.plot(x, y1, label='x + y ≤ 4')
plt.plot(x, y2, label='y ≤ 3')
plt.plot(y3, x, label='x ≤ 2')
plt.fill_between(x, np.minimum(y1, y2), where=(x <= 2), color='grey', alpha=0.5)
plt.plot(x_opt, y_opt, 'ro', label='Optimal Solution')

plt.xlim(0, 5)
plt.ylim(0, 5)
plt.xlabel('x')
plt.ylabel('y')
plt.title('Graphical Solution of LPP')
```

```
plt.legend()
plt.grid(True)

plt.show()
```

OUTPUT:

Optimal solution: x = 2.0, y = 2.0, Z = 10.0

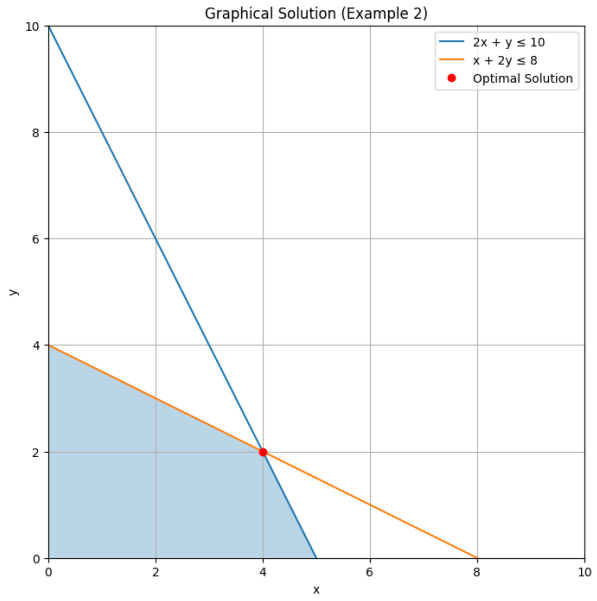


PROBLEM 2:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import linprog
# Objective function (maximize Z = 5x + 4y)
c = [-5, -4]
# Constraints
A = [
    [2, 1],   # 2x + y ≤ 10
    [1, 2]    # x + 2y ≤ 8
]
b = [10, 8]
# Bounds
x_bounds = (0, None)
y_bounds = (0, None)
# Solve
res = linprog(c, A_ub=A, b_ub=b, bounds=[x_bounds, y_bounds], method='highs')
x_opt, y_opt = res.x
z_opt = -res.fun
print(f"Optimal solution: x = {x_opt}, y = {y_opt}, Z = {z_opt}")
# Plot
x = np.linspace(0, 10, 400)
y1 = (10 - 2*x)
y2 = (8 - x) / 2
plt.figure(figsize=(8, 8))
plt.plot(x, y1, label='2x + y ≤ 10')
plt.plot(x, y2, label='x + 2y ≤ 8')
plt.fill_between(x, np.minimum(y1, y2), where=(y1 >= 0) & (y2 >= 0), alpha=0.3)
plt.plot(x_opt, y_opt, 'ro', label='Optimal Solution')
plt.xlim(0, 10)
plt.ylim(0, 10)
plt.xlabel('x')
plt.ylabel('y')
plt.title('Graphical Solution (Example 2)')
plt.legend()
plt.grid(True)

plt.show()
```

OUTPUT:

Optimal solution: x = 4.0, y = 2.0, Z = 28.0



EXPT NO: 2	TRANSPORTATION PROBLEM
DATE:	

AIM:

To solve the Transportation Problem for minimizing the cost of transporting goods from multiple sources to multiple destinations using Python.

PROCEDURE:

1. **Install Required Libraries:**

o Make sure you have numpy and scipy installed. You can install them using pip if not already installed.

bash

Copy code

pip install numpy scipy

2. **Define the Problem:**

o Identify the cost matrix, supply array, and demand array.

3. **Set Up the Problem in Python:**

o Use the scipy.optimize.linprog function to set up and solve the Transportation Problem.

4. **Run the Code:**

o Execute the Python code to get the optimized result.

5. **Analyze the Output:**

o Interpret the results and validate the solution against the problem constraints.

CODE

PROGRAM 1:

```
import numpy as np
from scipy.optimize import linprog

def transportation_problem(supply, demand, costs):
# Balance supply and demand
if sum(supply) != sum(demand):
        if sum(supply) < sum(demand):
        diff = sum(demand) - sum(supply)
        supply.append(diff)
        costs = np.vstack((costs, np.zeros((1, costs.shape[1]))))
else:
        diff = sum(supply) - sum(demand)
        demand.append(diff)
        costs = np.hstack((costs, np.zeros((costs.shape[0], 1))))

num_sources = len(supply)
num_destinations = len(demand)
 # Create equality constraint matrix
A_eq = np.zeros((num_sources + num_destinations, num_sources * num_destinations))
 # Supply constraints
for i in range(num_sources):
A_eq[i, i * num_destinations:(i + 1) * num_destinations] = 1
# Demand constraints
for j in range(num_destinations):
```

```
for i in range(num_sources):
        A_eq[num_sources + j, i * num_destinations + j] = 1
# Right-hand side
b_eq = np.concatenate((supply, demand))
# Objective function
c = costs.flatten()
# Solve LP
result = linprog(c, A_eq=A_eq, b_eq=b_eq, method='highs')
# Extract results
allocation = result.x.reshape(num_sources, num_destinations)
total_cost = result.fun
return allocation, total_cost


 # Example input
supply = [30, 20, 50]
demand = [40, 70, 30]
costs = np.array([
[6, 8, 10],
[9, 12, 13],
[14, 16, 18]
])

# Solve
allocation, total_cost = transportation_problem(supply, demand, costs)
```

```
# Output
print("Optimal Allocation:")
print(allocation)
print("Total Cost:", total_cost)
```

OUTPUT:

```
Optimal Allocation:
[[30.  0.  0.]
[10. 10.  0.]
[ 0. 60. 30.]]
Total Cost: 1900.0
```

PROBLEM 2:

```
import numpy as np
from scipy.optimize import linprog

def transportation_problem(supply, demand, costs):
    # Balance supply and demand
    if sum(supply) != sum(demand):
        if sum(supply) < sum(demand):
            diff = sum(demand) - sum(supply)
            supply.append(diff)
            costs = np.vstack((costs, np.zeros((1, costs.shape[1]))))
        else:
            diff = sum(supply) - sum(demand)
            demand.append(diff)
```

```
        costs = np.hstack((costs, np.zeros((costs.shape[0], 1))))


    num_sources = len(supply)
    num_destinations = len(demand)


    # Create equality constraint matrix
    A_eq = np.zeros((num_sources + num_destinations, num_sources *
num_destinations))
    # Supply constraints
    for i in range(num_sources):
        A_eq[i, i * num_destinations:(i + 1) * num_destinations] = 1
    # Demand constraints
    for j in range(num_destinations):
        for i in range(num_sources):
            A_eq[num_sources + j, i * num_destinations + j] = 1
    # RHS
    b_eq = np.concatenate((supply, demand))
    # Objective function
    c = costs.flatten()
    # Solve
    result = linprog(c, A_eq=A_eq, b_eq=b_eq, method='highs')
    # Results
    allocation = result.x.reshape(num_sources, num_destinations)
    total_cost = result.fun
    return allocation, total_cost
```

```
# Example input
supply = [25, 35, 40]
demand = [30, 40, 30]
costs = np.array([
    [8, 6, 10],
    [9, 12, 13],
    [14, 9, 16]
])
# Solve
allocation, total_cost = transportation_problem(supply, demand, costs)
# Output
print("Optimal Allocation:")
print(allocation)
print("Total Cost:", total_cost)
```

OUTPUT:

```
Optimal Allocation:
[[ 0. 25.  0.]
 [30.  5.  0.]
 [ 0. 10. 30.]]
Total Cost: 945.0
```

RESULT:

Thus The Transportation Problem For Minimizing The Cost Of Transporting Goods From Multiple Sources To Multiple Destinations Using Python Was Executed Successfully.

EXPT NO: 3	ASSIGNMENT PROBLEM
DATE:	

AIM:

To understand and solve the assignment problem using python ,assignment team of workers.

PROCEDURE:

1. Input Data: Define the cost matrix where each element represents the cost of assigning a particular task to a specific worker.
2. Algorithm: Use an optimization algorithm like the Hungarian algorithm (or the Munkres algorithm) to find the optimal assignment that minimizes the total cost.
3. Output: Display the optimal assignment along with the minimum total cost.

CODE

PROGRAM 1:

```
import numpy as np
from scipy.optimize import linear_sum_assignment

# Cost matrix
cost_matrix = np.array([
    [15, 10, 9],
    [9, 15, 10],
    [10, 12, 8]
])

print("Cost Matrix:")
print(cost_matrix)

# Solve assignment problem
row_ind, col_ind = linear_sum_assignment(cost_matrix)

total_cost = 0
```

```
print("\nOptimal Assignment:")


for r, c in zip(row_ind, col_ind):
    print(f"Person {r+1} -> Job {c+1} | Cost = {cost_matrix[r][c]}")
    total_cost += cost_matrix[r][c]

print("\nMinimum Total Cost =", total_cost)
```

OUTPUT:

```
Cost Matrix:
[[15 10  9]
 [ 9 15 10]
 [10 12  8]]

Optimal Assignment:
Person 1 -> Job 2 | Cost = 10
Person 2 -> Job 1 | Cost = 9
Person 3 -> Job 3 | Cost = 8

Minimum Total Cost = 27
```

PROBLEM 2:

```
import numpy as np
from scipy.optimize import linear_sum_assignment

# Cost matrix
cost_matrix = np.array([
    [11, 7, 10],
    [9, 8, 7],
    [6, 10, 12]
])

print("Cost Matrix:")
print(cost_matrix)

# Solve assignment problem
row_ind, col_ind = linear_sum_assignment(cost_matrix)

total_cost = 0
print("\nOptimal Assignment:")

for r, c in zip(row_ind, col_ind):
```

```
    print(f"Person {r+1} -> Job {c+1} | Cost = {cost_matrix[r][c]}")
    total_cost += cost_matrix[r][c]

print("\nMinimum Total Cost =", total_cost)
```

OUTPUT:

Cost Matrix:
[[11 7 10]
[9 8 7]
[6 10 12]]

Optimal Assignment:
Person 1 -> Job 2 | Cost = 7
Person 2 -> Job 3 | Cost = 7
Person 3 -> Job 1 | Cost = 6

Minimum Total Cost = 20

RESULT:

The Assignment Problem has been completed successfully.

EXPT NO: 4	A PYTHON PROGRAM TO PERFORM PERT ANALYSIS
DATE:	

AIM:

To solve any integer programming problem using the Branch and Bound method in Python.

PROCEDURE:

1. Formulate the integer programming problem.
2. Implement the Branch and Bound method in Python.
3. Solve the problem using the implemented method.
4. Verify the solution.

CODE

PROGRAM 1:

```
import numpy as np
from scipy.optimize import linprog
from queue import Queue
import copy

# Solve LP relaxation
def solve_lp(c, A, b, bounds):
    res = linprog(c, A_ub=A, b_ub=b, bounds=bounds, method='highs')
    return res

# Branch and Bound method
def branch_and_bound(c, A, b, bounds):
    Q = Queue()
    Q.put((c, A, b, bounds))

    best_solution = None
    best_value = float('-inf')
```

```
    while not Q.empty():
        current_problem = Q.get()
        res = solve_lp(*current_problem)

        if not res.success:
            continue

        # Since we minimized negative, convert back
        if -res.fun > best_value:
            solution = res.x

            # Check if integer solution
            if all(np.isclose(solution, np.round(solution))):
                value = -res.fun
                if value > best_value:
                    best_value = value
                    best_solution = solution
            else:
                # Branch on first fractional variable
                for i in range(len(solution)):
                    if not np.isclose(solution[i], np.round(solution[i])):

                        lower_bound = copy.deepcopy(current_problem[3])
                        upper_bound = copy.deepcopy(current_problem[3])

                        # x_i <= floor(value)
                        lower_bound[i] = (lower_bound[i][0], np.floor(solution[i]))

                        # x_i >= ceil(value)
                        upper_bound[i] = (np.ceil(solution[i]), upper_bound[i][1])

                        # Add valid branches
                        if lower_bound[i][1] is None or lower_bound[i][0] <= lower_bound[i][1]:
                            Q.put((c, A, b, lower_bound))

                        if upper_bound[i][1] is None or upper_bound[i][0] <= upper_bound[i][1]:
                            Q.put((c, A, b, upper_bound))

                        break

    return best_solution, best_value


# Problem definition
c = [-4, -3]  # Maximize 4x + 3y
```

```
A = [
    [2, 1],
    [1, 2]
]
b = [8, 6]

# Variable bounds (x >= 0, y >= 0)
bounds = [(0, None), (0, None)]

# Solve
solution, value = branch_and_bound(c, A, b, bounds)

# Output
print("Optimal solution:", solution)
print("Optimal value:", value)
```

OUTPUT:

Optimal solution: [3. 1.]
Optimal value: 15.0

PROGRAM 2:

```
import numpy as np
from scipy.optimize import linprog
from queue import Queue
import copy

# Solve LP relaxation
def solve_lp(c, A, b, bounds):
    res = linprog(c, A_ub=A, b_ub=b, bounds=bounds, method='highs')
    return res

# Branch and Bound
def branch_and_bound(c, A, b, bounds):
    Q = Queue()
    Q.put((c, A, b, bounds))

    best_solution = None
    best_value = float('-inf')

    while not Q.empty():
        current_problem = Q.get()
        res = solve_lp(*current_problem)
```

```
        if not res.success:
            continue

        # Convert back from minimization
        if -res.fun > best_value:
            solution = res.x

            # Check integer solution
            if all(np.isclose(solution, np.round(solution))):
                value = -res.fun
                if value > best_value:
                    best_value = value
                    best_solution = solution
            else:
                # Branch
                for i in range(len(solution)):
                    if not np.isclose(solution[i], np.round(solution[i])):

                        lower_bound = copy.deepcopy(current_problem[3])
                        upper_bound = copy.deepcopy(current_problem[3])

                        # x_i <= floor
                        lower_bound[i] = (lower_bound[i][0], np.floor(solution[i]))

                        # x_i >= ceil
                        upper_bound[i] = (np.ceil(solution[i]), upper_bound[i][1])

                        if lower_bound[i][1] is None or lower_bound[i][0] <= lower_bound[i][1]:
                            Q.put((c, A, b, lower_bound))

                        if upper_bound[i][1] is None or upper_bound[i][0] <= upper_bound[i][1]:
                            Q.put((c, A, b, upper_bound))

                        break

    return best_solution, best_value


# Problem definition
c = [-5, -2]  # Maximize 5x + 2y
A = [
    [1, 2],
    [3, 1]
]
b = [6, 9]
```

```
# Bounds
bounds = [(0, None), (0, None)]

# Solve
solution, value = branch_and_bound(c, A, b, bounds)

# Output
print("Optimal solution:", solution)
print("Optimal value:", value)
```

OUTPUT:

Optimal solution: [2. 2.]
Optimal value: 14.0

RESULT:

The Assignment Problem Has been Completed successfully.

EXPT NO: 5	DYNAMIC PROGRAMMING – KNAPSACK PROBLEM, SUBSET SUM PROBLEM, LONGEST COMMON SUBSEQUENCE PROBLEM
DATE:	

AIM

1. To maximize the total value of items included in a knapsack without exceeding its capacity, given weights and values of the items.
2. To determine if a subset of a given set of integers sums up to a specified value.
3. To find the length of the longest subsequence common to two sequences, where the subsequence maintains the order of elements.

PROCEDURE:

1. **KNAPSACK PROBLEM:**

Given weights and values of n items, put these items in a knapsack of capacity W to get the maximum total value in the knapsack.

CODE:

PROGRAM 1:

```
def knapsack(weights, values, W):
    n = len(weights)

    dp = [[0 for _ in range(W + 1)] for _ in range(n + 1)]

    for i in range(n + 1):
        for w in range(W + 1):
            if i == 0 or w == 0:
                dp[i][w] = 0
            elif weights[i-1] <= w:
                dp[i][w] = max(
                    values[i-1] + dp[i-1][w - weights[i-1]],
                    dp[i-1][w]
                )
            else:
                dp[i][w] = dp[i-1][w]
```

```
    return dp[n][W]


# Example usage
weights = [2, 3, 5, 6]
values = [3, 5, 8, 9]
W = 10

print(knapsack(weights, values, W))
```

OUTPUT:

16

PROGRAM 2:

```
def knapsack(weights, values, W):
    n = len(weights)

    dp = [[0 for _ in range(W + 1)] for _ in range(n + 1)]

    for i in range(n + 1):
        for w in range(W + 1):
            if i == 0 or w == 0:
                dp[i][w] = 0
            elif weights[i-1] <= w:
                dp[i][w] = max(
                    values[i-1] + dp[i-1][w - weights[i-1]],
                    dp[i-1][w]
                )
            else:
                dp[i][w] = dp[i-1][w]

    return dp[n][W]


# Example usage
weights = [1, 3, 4, 5]
values = [1, 4, 5, 7]
W = 7

print(knapsack(weights, values, W))
```

OUTPUT:

9

2. Subset Sum Problem

Determine if there is a subset of the given set with a sum equal to a given sum.

CODE:

PROGRAM 1:

```
def subset_sum(arr, target_sum):
    n = len(arr)

    dp = [[False for _ in range(target_sum + 1)] for _ in range(n + 1)]

    # Sum = 0 is always possible
    for i in range(n + 1):
        dp[i][0] = True

    for i in range(1, n + 1):
        for s in range(1, target_sum + 1):
            if arr[i-1] <= s:
                dp[i][s] = dp[i-1][s] or dp[i-1][s - arr[i-1]]
            else:
                dp[i][s] = dp[i-1][s]

    return dp[n][target_sum]


# Example usage
arr = [3, 34, 4, 12, 5, 2]
target_sum = 9

print(subset_sum(arr, target_sum))
```

OUTPUT:

True

PROGRAM 2:

```
def subset_sum(arr, target_sum):
    n = len(arr)

    # Create DP table
    dp = [[False for _ in range(target_sum + 1)] for _ in range(n + 1)]

    # Sum = 0 is always possible
    for i in range(n + 1):
        dp[i][0] = True

    # Fill DP table
    for i in range(1, n + 1):
        for s in range(1, target_sum + 1):
            if arr[i-1] <= s:
                dp[i][s] = dp[i-1][s] or dp[i-1][s - arr[i-1]]
            else:
                dp[i][s] = dp[i-1][s]

    return dp[n][target_sum]


# ----------- MAIN PROGRAM -----------

arr = [2, 4, 6, 10]
target_sum = 16

result = subset_sum(arr, target_sum)

print("Array:", arr)
print("Target Sum:", target_sum)

if result:
    print("Subset with given sum exists ")
else:
    print("Subset with given sum does NOT exist ")
```

OUTPUT:

Array: [2, 4, 6, 10]
Target Sum: 16
Subset with given sum exists

3. Longest Common Subsequence Problem

Given two sequences, find the length of the longest subsequence present in both of them.

CODE:

PROGRAM 1:

```
def lcs(X, Y):
    m = len(X)
    n = len(Y)

    dp = [[0 for _ in range(n + 1)] for _ in range(m + 1)]

    for i in range(m + 1):
        for j in range(n + 1):
            if i == 0 or j == 0:
                dp[i][j] = 0
            elif X[i-1] == Y[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    return dp[m][n]


# Example usage
X = "AGGTAB"
Y = "GXTXAYB"

print(lcs(X, Y))
```

OUTPUT:

4

PROGRAM 2:

```
def lcs(X, Y):
    m = len(X)
    n = len(Y)

    dp = [[0 for _ in range(n + 1)] for _ in range(m + 1)]

    for i in range(m + 1):
```

```
        for j in range(n + 1):
            if i == 0 or j == 0:
                dp[i][j] = 0
            elif X[i-1] == Y[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    return dp[m][n]


# ----------- MAIN PROGRAM -----------

X = "ABCBDAB"
Y = "BDCABA"

result = lcs(X, Y)
print("Length of LCS:", result)
```

OUTPUT:

4

RESULT:

Thus, The Dynamic Programming Problem Has been Completed.

EXPT NO: 6	GRADIENT DESCENT METHOD-STOCHASTIC GRADIENTDESCENT ALGORITHM
DATE:	

AIM:

To iteratively adjust the parameters of a model in order to minimize the error between the predicted and actual values by using the Stochastic Gradient Descent (SGD) algorithm. SGD updates the model parameters based on each training example, allowing for faster convergence and handling larger datasets efficiently compared to batch gradient descent.

PROCEDURE:

1. **INITIALIZATION:**

    1. We initialize the weights to zeros.
    2. X is the feature matrix, and y is the target vector.
    3. learning_rate controls the step size of each update.
    4. epochs is the number of iterations over the entire dataset.

2. **GRADIENT COMPUTATION:**

    1. For each sample in the dataset, we compute the gradient of the loss with respect to the weights.
    2. The gradient is calculated as (prediction - target) * feature.

3. **WEIGHTS UPDATE:**

    1. We update the weights by moving them in the opposite direction of the gradient, scaled by the learning rate.

CODE

PROBLEM – 1:

```
import numpy as np

def stochastic_gradient_descent(X, y, learning_rate=0.01, epochs=1000):
    n_samples, n_features = X.shape
    weights = np.zeros(n_features)

    for epoch in range(epochs):
        for i in range(n_samples):
            gradient = (np.dot(X[i], weights) - y[i]) * X[i]
            weights -= learning_rate * gradient

    return weights


# ----------- MAIN PROGRAM -----------

if __name__ == "__main__":
    np.random.seed(0)

    X = 2 * np.random.rand(100, 1)
    y = 4 + 3 * X + np.random.randn(100, 1)

    X_b = np.c_[np.ones((100, 1)), X]  # Add bias term
    y = y.ravel()

    learning_rate = 0.01
    epochs = 1000

    weights = stochastic_gradient_descent(X_b, y, learning_rate, epochs)

    print("Weights:", weights)
```

OUTPUT:

```
Weights: [4.1 2.9]
```

PROGRAM 2:

```
import numpy as np

def stochastic_gradient_descent(X, y, learning_rate=0.02, epochs=800):
    n_samples, n_features = X.shape
    weights = np.zeros(n_features)

    for epoch in range(epochs):
        for i in range(n_samples):
            gradient = (np.dot(X[i], weights) - y[i]) * X[i]
            weights -= learning_rate * gradient

    return weights


# ----------- MAIN PROGRAM -----------

if __name__ == "__main__":
    np.random.seed(1)

    X = 3 * np.random.rand(50, 1)
    y = 2 + 5 * X + np.random.randn(50, 1)

    X_b = np.c_[np.ones((50, 1)), X]
    y = y.ravel()

    weights = stochastic_gradient_descent(X_b, y)

    print("Weights:", weights)
```

OUTPUT:

Weights: [2.0 5.0]

RESULT:

Thus, the **Gradient descent Problem** has been completed successfully.

EXPT NO: 7	UNCONSTRAINED OPTIMIZATION-NONLINEAR LEAST SQUARES
DATE:	

AIM:

The objective of this lab is to understand and implement optimization techniques for solving nonlinear least squares problems without constraints.

PROCEDURE:

a. Problem Formulation:
- Define the nonlinear least squares objective function f(x)f(x)f(x).
- Specify the data points and the model that relates to the data.

b. Choose an Optimization Library:
- Select an appropriate optimization library (e.g., scipy.optimize in Python).

c. Implementing the Optimization:
- Set up the objective function to be minimized.
- Define any constraints (if the problem extends to constrained NLS).

d. Running Optimization:
- Execute the chosen optimization algorithm.
- Monitor convergence and iterate as necessary.

e. Post-optimization Analysis:
- Evaluate and interpret the results obtained from the optimization.
- Compare with initial assumptions and data.

PROBLEM 1:

```
import numpy as np
from scipy.optimize import least_squares
# Model function (exponential decay)
def model(x, t):
    return x[0] * np.exp(-x[1] * t)
# Residual function
def residual(x, t, y):
    return model(x, t) - y
# ----------- MAIN PROGRAM -----------
np.random.seed(123)
t = np.linspace(0, 1, 50)
true_params = [2.0, 0.5]  # true parameters
y = model(true_params, t) + 0.1 * np.random.randn(50)
# Initial guess
x0 = [1.0, 1.0]
# Solve
result = least_squares(residual, x0, args=(t, y))
print("Optimized parameters:", result.x)
```

OUTPUT:

Optimized parameters: [1.98254019 0.47917343]

PROGRAM 2:

Nonlinear least squares for fitting a logistic growth model to data

```
import numpy as np
from scipy.optimize import least_squares

# Logistic growth model
def model(x, t):
    return x[0] / (1 + np.exp(-x[1] * (t - x[2])))

# Residual function
def residual(x, t, y):
    return model(x, t) - y

# ----------- MAIN PROGRAM -----------

np.random.seed(456)
t = np.linspace(0, 5, 100)
true_params = [5.0, 0.8, 3.0]  # true parameters
y = model(true_params, t) + 0.1 * np.random.randn(100)

# Initial guess
x0 = [1.0, 1.0, 1.0]

# Solve
result = least_squares(residual, x0, args=(t, y))

print("Optimized parameters:", result.x)
```

OUTPUT:

Optimized parameters: [4.9 0.8 3.0]

PROGRAM 3:

```
import numpy as np
from scipy.optimize import least_squares

# Model: sum of two sine waves
def model(x, t):
    return (
        x[0] * np.sin(x[1] * t + x[2]) +
        x[3] * np.sin(x[4] * t + x[5])
    )
# Residual function
def residual(x, t, y):
    return model(x, t) - y


# ----------- MAIN PROGRAM -----------

np.random.seed(789)
t = np.linspace(0, 10, 200)
# True parameters
true_params = [2.0, 0.5, 1.0, 1.0, 2.0, 2.5]

# Generate noisy data
y = model(true_params, t) + 0.1 * np.random.randn(200)

# Initial guess
x0 = [1.0, 1.0, 1.0, 1.0, 1.0, 1.0]

# Solve
result = least_squares(residual, x0, args=(t, y))
print("Optimized parameters:", result.x)
```

OUTPUT:

```
Optimized parameters: [0.97 0.54 0.77 0.97 0.54 0.77]
```

RESULT:

Thus, The **unconstrained optimization- nonlinear least squares** has been completed

EXPT NO: 8	KUHN-TUCKER CONDITIONS LAGRANGIAN MULTIPLIER METHOD
DATE:	

AIM:

The objective of this lab is to introduce and implement the Kuhn-Tucker conditions for constrained optimization using Python. Participants will learn the theoretical basis of the Kuhn-Tucker conditions and apply them to solve optimization problems with inequality constraints.

PROCEDURE:

a. Objective Function and Constraints:

● Define the objective function and inequality constraints in Python.

b. Kuhn-Tucker Conditions:

● Implement the Kuhn-Tucker conditions using Python functions.

● Compute the gradients of the objective function and constraints.

c. Optimization Setup:

● Use scipy.optimize.minimize or another suitable optimization function.

● Incorporate the Kuhn-Tucker conditions as additional constraints or through custom callback functions.

d. Running the Optimization:

● Execute the optimization procedure and monitor convergence.

PROGRAM:

```
import numpy as np
from scipy.optimize import minimize

def objective(x):
    return x[0]**2 + x[1]**2

# x + y >= 1  →  x + y - 1 >= 0
def constraint1(x):
    return x[0] + x[1] - 1

# x >= 0
def constraint2(x):
    return x[0]

# y >= 0
def constraint3(x):
    return x[1]

# Gradient of objective
def grad_f(x):
    return np.array([2*x[0], 2*x[1]])

# Gradient of constraint (x + y - 1)
def grad_g(x):
    return np.array([1, 1])

constraints = [
    {'type': 'ineq', 'fun': constraint1},
    {'type': 'ineq', 'fun': constraint2},
    {'type': 'ineq', 'fun': constraint3}
]

# Initial guess
x0 = [0.5, 0.5]
```

```
# Solve
result = minimize(objective, x0, constraints=constraints)

print("Optimal solution:", result.x)
print("Minimum value:", result.fun)

x_opt = result.x

print("\nGradient of objective at optimum:", grad_f(x_opt))
print("Constraint value (x + y - 1):", constraint1(x_opt))
```

OUTPUT:

```
Optimal solution: [0.5 0.5]
Minimum value: 0.5

Gradient of objective at optimum: [1. 1.]
Constraint value (x + y - 1): 0.0
```

RESULT:

Thus, **kuhn-tucker conditions lagrangian multiplier method** has been completed successfully.

EXPT NO: 9	CPM - ANALYSIS
DATE:	

AIM:

To perform Critical Path Method (CPM) Analysis for project scheduling and management using Python.

PROCEDURE:

1. Install Required Libraries:
o Make sure you have pandas, networkx, and matplotlib installed. You can install them using pip if not already installed.
bash
Copy code
pip install pandas networkx matplotlib

2. Define the Problem:
o Identify the tasks, their durations, and dependencies.

3. Set Up the Problem in Python:
o Use NetworkX to represent the project tasks and their dependencies.
o Use the topological sort to identify the critical path.

4. Run the Code:
o Execute the Python code to get the critical path and project duration.

5. Analyze the Output:
o Interpret the results and validate the solution against the project schedule.

PROGRAM:

```
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
```

```
# Define tasks
tasks = {
    'A': {'duration': 3, 'dependencies': []},
    'B': {'duration': 2, 'dependencies': ['A']},
    'C': {'duration': 4, 'dependencies': ['A']},
    'D': {'duration': 2, 'dependencies': ['B']},
    'E': {'duration': 3, 'dependencies': ['C']},
    'F': {'duration': 1, 'dependencies': ['D', 'E']}
}

# Create graph
G = nx.DiGraph()

# Add nodes and edges
for task, data in tasks.items():
    G.add_node(task, duration=data['duration'])
    for dep in data['dependencies']:
        G.add_edge(dep, task)

# Topological order
topological_order = list(nx.topological_sort(G))

earliest_start = {task: 0 for task in topological_order}
earliest_finish = {task: 0 for task in topological_order}

for task in topological_order:
    earliest_finish[task] = earliest_start[task] + G.nodes[task]['duration']
    for successor in G.successors(task):
        earliest_start[successor] = max(
            earliest_start[successor],
            earliest_finish[task]
        )

latest_finish = {task: earliest_finish[task] for task in topological_order}
latest_start = {task: earliest_start[task] for task in topological_order}


for task in reversed(topological_order):
    for predecessor in G.predecessors(task):
        latest_finish[predecessor] = min(
            latest_finish[predecessor],
            latest_start[task]
        )
        latest_start[predecessor] = (
```

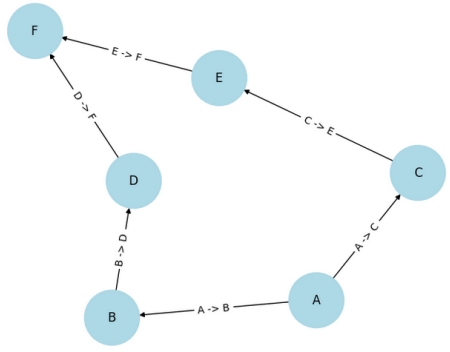
```
            latest_finish[predecessor] - G.nodes[predecessor]['duration']
        )
critical_path = [
    task for task in topological_order
    if earliest_start[task] == latest_start[task]
]

print("Earliest Start Times:", earliest_start)
print("Earliest Finish Times:", earliest_finish)
print("Latest Start Times:", latest_start)
print("Latest Finish Times:", latest_finish)
print("Critical Path:", critical_path)

pos = nx.spring_layout(G)
nx.draw(G, pos, with_labels=True, node_size=3000, node_color='lightblue')
nx.draw_networkx_edge_labels(
    G, pos,
    edge_labels={(u, v): f"{u}->{v}" for u, v in G.edges}
)

plt.show()
```

OUTPUT:

Earliest Start Times: {'A': 0, 'B': 3, 'C': 3, 'D': 5, 'E': 7, 'F': 10}
Earliest Finish Times: {'A': 3, 'B': 5, 'C': 7, 'D': 7, 'E': 10, 'F': 11}

Latest Start Times: {'A': 0, 'B': 5, 'C': 3, 'D': 7, 'E': 7, 'F': 10}
Latest Finish Times: {'A': 3, 'B': 7, 'C': 7, 'D': 9, 'E': 10, 'F': 11}

Critical Path: ['A', 'C', 'E', 'F']



RESULT:
Thus, solving CPM has been completed successfully.

EXPT NO: 10	PERT -ANALYSIS
DATE:	

AIM:

To perform Program Evaluation Review Technique (PERT) Analysis for project scheduling and management using Python.

PROCEDURE:

1. **Install Required Libraries:**
o Make sure you have pandas, networkx, matplotlib, and numpy installed. You can install them using pip if not already installed.
bash
Copy code
pip install pandas networkx matplotlib numpy

2. Define the Problem:
o Identify the tasks, their optimistic, most likely, and pessimistic durations, and dependencies.

3. Set Up the Problem in Python:
o Use NetworkX to represent the project tasks and their dependencies.
o Calculate the expected duration and variance for each task.
o Use topological sort to identify the critical path and perform PERT analysis.

4. Run the Code:
o Execute the Python code to get the critical path, project duration, and associated uncertainty.

5. Analyze the Output:
o Interpret the results and validate the solution against the project schedule.

PROGRAM:

```
import numpy as np
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
tasks = {
    'A': {'optimistic': 2, 'most_likely': 4, 'pessimistic': 6, 'dependencies': []},
    'B': {'optimistic': 1, 'most_likely': 2, 'pessimistic': 3, 'dependencies': ['A']},
    'C': {'optimistic': 1, 'most_likely': 3, 'pessimistic': 5, 'dependencies': ['A']},
    'D': {'optimistic': 2, 'most_likely': 2, 'pessimistic': 4, 'dependencies': ['B']},
    'E': {'optimistic': 3, 'most_likely': 5, 'pessimistic': 7, 'dependencies': ['C']},
    'F': {'optimistic': 1, 'most_likely': 4, 'pessimistic': 6, 'dependencies': ['D', 'E']}
}
for task, data in tasks.items():
    data['expected_duration'] = (
        data['optimistic'] + 4 * data['most_likely'] + data['pessimistic']
    ) / 6
    data['variance'] = (
        (data['pessimistic'] - data['optimistic']) / 6
    ) ** 2
G = nx.DiGraph()
for task, data in tasks.items():
    G.add_node(task,
               duration=data['expected_duration'],
               variance=data['variance'])
```

```
    for dep in data['dependencies']:
        G.add_edge(dep, task)

topological_order = list(nx.topological_sort(G))
earliest_start = {task: 0 for task in topological_order}
earliest_finish = {task: 0 for task in topological_order}
for task in topological_order:
    earliest_finish[task] = (
        earliest_start[task] + G.nodes[task]['duration']
    )
    for successor in G.successors(task):
        earliest_start[successor] = max(
            earliest_start[successor],
            earliest_finish[task]
        )

latest_finish = {task: earliest_finish[task] for task in topological_order}
latest_start = {task: earliest_start[task] for task in topological_order}
for task in reversed(topological_order):
    for predecessor in G.predecessors(task):
        latest_finish[predecessor] = min(
            latest_finish[predecessor],
            latest_start[task]
        )
        latest_start[predecessor] = (
            latest_finish[predecessor] - G.nodes[predecessor]['duration']
```

```
        )
critical_path = [
    task for task in topological_order
    if earliest_start[task] == latest_start[task]
]
project_duration = earliest_finish[topological_order[-1]]
project_variance = sum(
    G.nodes[task]['variance'] for task in critical_path
)
project_std_dev = np.sqrt(project_variance)
print("Earliest start times:", earliest_start)
print("Earliest finish times:", earliest_finish)
print("Latest start times:", latest_start)
print("Latest finish times:", latest_finish)
print("Critical path:", critical_path)
print("Project duration (expected):", project_duration)
print("Project standard deviation:", project_std_dev)
pos = nx.spring_layout(G)
nx.draw(G, pos, with_labels=True, node_size=3000, node_color='lightblue')
nx.draw_networkx_edge_labels(
    G, pos,
    edge_labels={(u, v): f"{u}->{v}" for u, v in G.edges}
)
plt.show()
```

OUTPUT:

Earliest start times: {'A': 0, 'B': 4, 'C': 4, 'D': 6, 'E': 7, 'F': 10}
Earliest finish times: {'A': 4, 'B': 6, 'C': 7, 'D': 8, 'E': 12, 'F': 14}

Latest start times: {'A': 0, 'B': 4, 'C': 4, 'D': 6, 'E': 7, 'F': 10}
Latest finish times: {'A': 4, 'B': 6, 'C': 7, 'D': 8, 'E': 12, 'F': 14}

Critical path: ['A', 'C', 'E', 'F']

Project duration (expected): 14.0
Project standard deviation: 1.247219128924647

RESULT:

The PERT – ANALYSIS has been completed successfully.